Informe Laboratorio 3

Sección 3

Ihan San Martin
e-mail: ihan.sanmartin@mail.udp.cl

14 de octubre de 2026

Índice

1. Descripción de actividades **2**

2. Desarrollo de actividades según criterio de rúbrica **3**
2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio 4
2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión 9
2.3. Genera el hash de la contraseña desde la consola del navegador 11
2.4. Intercepta el tráfico login con BurpSuite 12
2.5. Realiza el intento de login 12
2.6. Identifica las políticas de privacidad o seguridad 12
2.7. Demuestra 4 conclusiones sobre la seguridad 12

1. Descripción de actividades

Su objetivo será auditar la implementación de algoritmos hash aplicados a contraseñas en páginas web desde el lado del cliente, así como evaluar la efectividad de estas medidas contra ataques de tipo Pass the Hash (PtH). Para llevar a cabo esta auditoría, deberá registrarse en un sitio web y crear una cuenta, ingresando una contraseña específica para realizar las pruebas.

Al concluir la tarea, es importante que modifique su contraseña por una diferente para garantizar su seguridad.

Dado que la cantidad de sitios chilenos que utilizan hash es limitada, se permite realizar esta tarea en cualquier sitio web a nivel mundial. En este sentido, realice las siguientes actividades:

- Identificación del algoritmo de hash utilizado para las contraseñas al momento del registro en el sitio.
- Identificación del algoritmo de hash utilizado para las contraseñas al momento de iniciar sesión.
- Generación del hash de la contraseña desde la consola del navegador, partiendo de la contraseña en texto plano.
- Interceptación del tráfico de login utilizando BurpSuite desde su equipo.
- Realización de un intento de login, modificando una contraseña incorrecta por el hash obtenido en el punto anterior.
- Descripción de las políticas de privacidad o seguridad relacionadas con las contraseñas, incluyendo un enlace a las mismas.
- Cuatro conclusiones sobre la seguridad o vulnerabilidad de la implementación observada.

2. Desarrollo de actividades según criterio de rúbrica

Para el desarrollo de esta experiencia de laboratorio se utilizó la página web ***mega.io***, con un poco de investigación se descubrió que este servicio de almacenamiento en la nube y file-sharing hace **hashing** en el lado del cliente, esto debido a su mecanismo de **zero-knowledge**, que básicamente significa que toda la encriptación y desencriptación se hace en el dispositivo del usuario, de esa forma ni siquiera MEGA puede acceder a tus archivos y obviamente la contraseña.

Figura 1: Logotipo de mega.io/mega.nz

Antes de comenzar con cada punto de la rúbrica, se debe empezar por crear una cuenta, monitoreando las peticiones **HTTP** que se hacen para analizar qué sucede internamente en la página. Para esta prueba se utilizó un correo electrónico personal con la contraseña "@criptoyseg". Se hacen variadas peticiones del tipo **POST**, pero la más importante fue la siguiente.


```
▼ 0: {...}
    a: "uc2"
    crv: "1E6VckQ9Svz4vpd45cQMMA"
    hak: "RfZ1eCd35JaK6jCIvhc-iw"
    k: "fB1_rPQtfs7RqVICTIjeJA"
    m: "aWhhbnNhbm1hcnRpbi5tQGdtYWIsLmNvbQ"
    n: "cHJ1ZWJhbGFiY3JpcHRvIA"
    v: 2
```


Figura 2: Petición de registro con datos encriptados

Esta contiene distintas cadenas que se derivan de la contraseña, correo, un salt generado aleatoriamente, etc.

2.1. Identifica el algoritmo de hash utilizado al momento de registrarse en el sitio

Para identificar el algoritmo utilizado, hizo falta entrar en los archivos de la página, específicamente al script que maneja el inicio de sesión y registro de cuentas. Específicamente se analizó un script sin nombre fijo que va variando por cada vez que se recarga la página, es del tipo "0c09f628-6b90-4875-99c0-d1002144fee7".

Para acceder por cuenta propia a los archivos de la página se siguieron los siguientes pasos con el navegador Firefox:

- F12 > Debugger > Main Thread > blob:// > https://mega.nz > [SCRIPT]

Lo primero que se buscó fue la función que maneja el registro de usuarios y se encontró el siguiente bloque de código.

```
1  startRegistration: function(firstName, lastName, email, password,
   ↪  fromProPage, completeCallback, skipLoading) {
2          'use strict';
3
4          if (this.sendEmailRequestParams) {
5              console.error('startRegistration blocked, ongoing.');
6              return;
7          }
8          this.sendEmailRequestParams = true;
9
10         if (!skipLoading) {
11             // Show loading dialog
12             loadingDialog.show();
13         }
14
15         // First create an ephemeral account and the Master Key (to be
           ↪  removed at a later date)
16         security.register.createEphemeralAccount(function() {
17
18             // Derive the Client Random Value, Encrypted Master Key and
               ↪  Hashed Authentication Key
19             security.deriveKeysFromPassword(password, u_k)
20                 .then(({clientRandomValueBytes, encryptedMasterKeyArray32,
                   ↪  hashedAuthenticationKeyBytes}) => {
21
```

```
22                      // Encode parameters to Base64 before sending to the
                        ↪ API
23                      const req = {
24                          a: 'uc2',
25                          n: base64urlencode(to8(firstName + ' ' + lastName)),
                            ↪ // Name (used just for the email)
26                          m: base64urlencode(email),
                            ↪ // Email
27                          crv: ab_to_base64(clientRandomValueBytes),
                            ↪ // Client Random Value
28                          k: a32_to_base64(encryptedMasterKeyArray32),
                            ↪ // Encrypted Master Key
29                          hak: ab_to_base64(hashedAuthenticationKeyBytes),
                            ↪ // Hashed Authentication Key
30                          v: 2
                            ↪ // Version of this protocol
31                      };
32
33                      // If this was a registration from the Pro page
34                      if (fromProPage === true) {
35                          req.p = 1;
36                      }
37
38                      // Send signup link email
39                      security.register.sendSignupLink(req, firstName,
                        ↪ lastName, email, completeCallback);
40                  });
41          });
42      }
```

Este bloque nos da la pista de donde viene el campo hak (linea 29) de la petición POST que se encontró, ya que esta función se encarga de construir la petición con todos los campos necesarios, así que luego de esto se va buscando inversamente cómo se construye la variable **hashedAuthenticationKeyBytes**.

Buscando en el código se encontró el siguiente bloque de la función **deriveKeysFromPassword**.

```
1
2       deriveKeysFromPassword: async function(password, masterKeyArray32) {
3           'use strict';
4
5           // Create the 128 bit (16 byte) Client Random Value and Salt
6           var saltLengthInBytes = security.saltLengthInBits / 8;
```

```
7           var clientRandomValueBytes = crypto.getRandomValues(new
            ↪ Uint8Array(saltLengthInBytes));
8           var saltBytes = security.createSalt(clientRandomValueBytes);
9
10          // Trim the password and convert it from ASCII/UTF-8 to a byte
            ↪ array
11          var passwordTrimmed = $.trim(password);
12          var passwordBytes = security.stringToByteArray(passwordTrimmed);
13
14          // The number of iterations for the PPF and desired length in bits
            ↪ of the derived key
15          var iterations = security.numOfIterations;
16          var derivedKeyLength = security.derivedKeyLengthInBits;
17
18          // Run the PPF
19          const derivedKeyBytes = await security.deriveKey(saltBytes,
            ↪ passwordBytes, iterations, derivedKeyLength);
20
21          // Get the first 16 bytes as the Encryption Key and the next 16
            ↪ bytes as the Authentication Key
22          const derivedEncryptionKeyBytes = derivedKeyBytes.subarray(0, 16);
23          const derivedAuthenticationKeyBytes = derivedKeyBytes.subarray(16,
            ↪ 32);
24
25          // Get a hash of the Authentication Key which the API will use for
            ↪ authentication at login time
26          let hashedAuthenticationKeyBytes =
            ↪ asmCrypto.SHA256.bytes(derivedAuthenticationKeyBytes);
27
28          // Keep only the first 128 bits (16 bytes) of the Hashed
            ↪ Authentication Key
29          hashedAuthenticationKeyBytes =
            ↪ hashedAuthenticationKeyBytes.subarray(0, 16);
30
31          // Convert the Derived Encryption Key to a big endian array of 32
            ↪ bytes, then encrypt the Master Key
32          const derivedEncryptionKeyArray32 =
            ↪ base64_to_a32(ab_to_base64(derivedEncryptionKeyBytes));
33          const cipherObject = new
            ↪ sjcl.cipher.aes(derivedEncryptionKeyArray32);
34          const encryptedMasterKeyArray32 = encrypt_key(cipherObject,
            ↪ masterKeyArray32);
35
```

```
36          // Pass the Client Random Value, Encrypted Master Key and Hashed
            ↪ Authentication Key to the calling function
37          return {
38              clientRandomValueBytes,
39              encryptedMasterKeyArray32,
40              hashedAuthenticationKeyBytes,
41              derivedAuthenticationKeyBytes
42          };
43      }
44
```

Esta función se encarga de calcular cada valor de los campos de la petición POST, el que nos interesa es el hak (hashed authentication key) y como se puede ver en la linea 26 del bloque anterior, hashedAuthenticationKeyBytes se calcula con el hash SHA-256. Pero esto no es todo, también en la linea 19, se puede ver como se usa la función **deriveKey**, con varios parámetros importantes, como el salt, la contraseña, el número de iteraciones que se define en otra parte del código como 100.000, y un largo de bits, entonces investigando más en el código se encontraron las funciones "Padre" de la encriptación de esta página.

```
1
2       deriveKey: async function(saltBytes, passwordBytes, iterations,
        ↪ derivedKeyLength) {
3           'use strict';
4
5           // If Web Crypto method supported, use that as it's nearly as fast
            ↪ as native
6           if (window.crypto && window.crypto.subtle && !window.is_microsoft) {
7               return security.deriveKeyWithWebCrypto(saltBytes, passwordBytes,
                ↪ iterations, derivedKeyLength);
8           }
9
10          // Otherwise, use asmCrypto which is the next fastest
11          return security.deriveKeyWithAsmCrypto(saltBytes, passwordBytes,
            ↪ iterations, derivedKeyLength);
12      },
13
14      /**
15       * Derive the key using the Web Crypto API
16       * @param {Uint8Array} saltBytes The salt as a byte array
17       * @param {Uint8Array} passwordBytes The password as a byte array
18       * @param {Number} iterations The cost factor / number of iterations
         ↪ of the PPF to perform
19       * @param {Number} derivedKeyLength The length of the derived key to
         ↪ create
```

```
20      */
21      deriveKeyWithWebCrypto: async function(saltBytes, passwordBytes,
        ↪  iterations, derivedKeyLength) {
22          'use strict';
23
24          // Import the password as the key
25          return crypto.subtle.importKey('raw', passwordBytes, 'PBKDF2',
            ↪  false, ['deriveBits'])
26              .then((key) => {
27
28                  // Required PBKDF2 parameters
29                  var params = {
30                      name: 'PBKDF2',
31                      hash: 'SHA-512',
32                      salt: saltBytes,
33                      iterations: iterations
34                  };
35
36                  // Derive bits using the algorithm
37                  return crypto.subtle.deriveBits(params, key,
                    ↪  derivedKeyLength);
38              })
39              .then((derivedKeyArrayBuffer) => {
40
41                  // Convert to a byte array
42                  // Pass the derived key to the callback
43                  return new Uint8Array(derivedKeyArrayBuffer);
44              });
45      },
46
47      /**
48       * Derive the key using asmCrypto
49       * @param {Uint8Array} saltBytes The salt as a byte array
50       * @param {Uint8Array} passwordBytes The password as a byte array
51       * @param {Number} iterations The cost factor / number of iterations
         ↪  of the PPF to perform
52       * @param {Number} derivedKeyLength The length of the derived key to
         ↪  create
53       */
54      deriveKeyWithAsmCrypto: async function(saltBytes, passwordBytes,
        ↪  iterations, derivedKeyLength) {
55          'use strict';
56
```

```
57          // Convert the desired derived key length to bytes and derive the
            ↪ key
58          var keyLengthBytes = derivedKeyLength / 8;
59
60          // Pass the derived key to the callback
61          return asmCrypto.PBKDF2_HMAC_SHA512.bytes(passwordBytes, saltBytes,
            ↪ iterations, keyLengthBytes);
62      }
63
```

Como se puede observar en el bloque de código, las funciones deriveKeyWithWebCrypto y deriveKeyWithAsmCrypto utilizan el algoritmo PBKDF2 con HMAC-SHA512, que es un algoritmo de derivación de claves que aplica múltiples iteraciones de hashing para aumentar la seguridad de las contraseñas. En este caso, se utiliza un salt generado aleatoriamente y 100.000 iteraciones, lo que hace que el proceso de hashing sea más lento y resistente a ataques de fuerza bruta.

Este conjunto de funciones calculan la clave derivada maestra de 32 bytes, la cual se divide a la mitad, y es sólo una mitad a la cual se le aplica un hash SHA-256 para generar el parámetro hak.

2.2. Identifica el algoritmo de hash utilizado al momento de iniciar sesión

Para el caso del login el flujo es un poco distinto, y se puede evidenciar en el siguiente bloque de código, que es la función que maneja el inicio de sesión.

```
1
2       startLogin: function(email, password, pinCode, rememberMe, salt,
        ↪ loginCompleteCallback) {
3
4           'use strict';
5
6           // Convert the salt and password to byte arrays
7           var saltArrayBuffer = base64_to_ab(salt);
8           var saltBytes = new Uint8Array(saltArrayBuffer);
9           var passwordBytes = security.stringToByteArray(password);
10
11          // The number of iterations for the PPF and desired length in bits
            ↪ of the derived key
12          var iterations = security.numOfIterations;
13          var derivedKeyLength = security.derivedKeyLengthInBits;
14
15          // Set the callback to run after login is complete
```

```
16          security.login.loginCompleteCallback = loginCompleteCallback;
17
18          // Store the remember-me flag for any direct cals to this function
            ↪  (e.g. atomicSignIn)
19          security.login.rememberMe = rememberMe !== false;
20
21          // Run the PPF
22          return security.deriveKey(saltBytes, passwordBytes, iterations,
            ↪  derivedKeyLength).then((derivedKeyBytes) => {
23
24              // Get the first 16 bytes as the Encryption Key and the next 16
                ↪  bytes as the Authentication Key
25              var derivedEncryptionKeyBytes = derivedKeyBytes.subarray(0, 16);
26              var derivedAuthenticationKeyBytes = derivedKeyBytes.subarray(16,
                ↪  32);
27              var authenticationKeyBase64 =
                ↪  ab_to_base64(derivedAuthenticationKeyBytes);
28
29              // Convert the Derived Encryption Key to a big endian array of
                ↪  32 bit values for decrypting the Master Key
30              var derivedEncryptionKeyArray32 =
                ↪  base64_to_a32(ab_to_base64(derivedEncryptionKeyBytes));
31
32              // Authenticate with the API
33              return security.login.sendAuthenticationKey(
34                  email,
35                  pinCode,
36                  authenticationKeyBase64,
37                  derivedEncryptionKeyArray32
38              );
39          }).catch((ex) => {
40              // @todo revamp and ditch the callbacks hell... {ref:
                ↪  atomicSignIn}
41              if (security.login.loginCompleteCallback) {
42                  return security.login.loginCompleteCallback(ex);
43              }
44              throw ex;
45          });
46      }
47
```

En las lineas 32 a 38 se puede observar que la authenticación se hace a través de la API, esto significa que ya no es del lado del cliente como en el registro, pero para verificar el acceso se deben usar los mismo algoritmos que para el registro para obtener el hash y

compararlo con el que se encuentra guardado en la base de datos, es decir se utiliza PBKDF2 con HMAC-SHA512 y SHA-256.

2.3. Genera el hash de la contraseña desde la consola del navegador

Para comprobar el funcionamiento del hashing por lado del cliente, se puede generar el hash de la contraseña desde la consola del navegador utilizando el siguiente código:

```
// 1. Convertir Base64URL a Base64 estándar (reemplazando caracteres y añadiendo padding)
var crvString = "1E6VckQ9Svz4vpd45cQMMA";
var base64Standard = crvString.replace(/-/g, '+').replace(/_/g, '/') + "==";

// 2. Decodificar manualmente a un arreglo estricto (Uint8Array)
var binaryString = atob(base64Standard);
var clientRandomValueBytes = new Uint8Array(binaryString.length);
for (var i = 0; i < binaryString.length; i++) {
    clientRandomValueBytes[i] = binaryString.charCodeAt(i);
}

// Verificación del paso: Debería imprimir exactamente "16". Si es así, la conversión fue un éxito.
console.log("Tamaño de los bytes del CRV:", clientRandomValueBytes.length);

// 3. Crear la sal con el arreglo corregido
var saltBytes = security.createSalt(clientRandomValueBytes);

// 4. Prepara tu contraseña exacta
var passwordTrimmed = $.trim("@criptoyseg"); // ¡No olvides cambiar esto!
var passwordBytes = security.stringToByteArray(passwordTrimmed);

// 5. Ejecutar la función de derivación
security.deriveKey(saltBytes, passwordBytes, security.numOfIterations, security.derivedKeyLengthInBits).then(derivedKeyBytes => {

    // Extraer la segunda mitad y aplicar SHA256
    var derivedAuthenticationKeyBytes = derivedKeyBytes.subarray(16, 32);
    var hashedAuthenticationKeyBytes = asmCrypto.SHA256.bytes(derivedAuthenticationKeyBytes).subarray(0, 16);

    // Imprimir los resultados finales
    console.log("HAK Calculado:", ab_to_base64(hashedAuthenticationKeyBytes));
    console.log("HAK Esperado :", "RfZ1eCd35JaK6jCIvhc-iw");
}).catch(err => console.error("Error en derivación:", err));
```

Figura 3: Código para generar el hash de la contraseña en la consola del navegador

Primero se necesita saber los valores de la petición por dos razones:

- Obtener el salt exacto del usuario que es necesario para el hash final.
- Saber cuál es el valor de hak para comparar el resultado calculado en la consola, con el hak real del usuario.

Luego se debe modificar el campo de la contraseña en el código con la que usamos de prueba, que en este caso fue “@criptoyseg” y finalmente al ejecutar el código, se obtuvo el siguiente resultado.

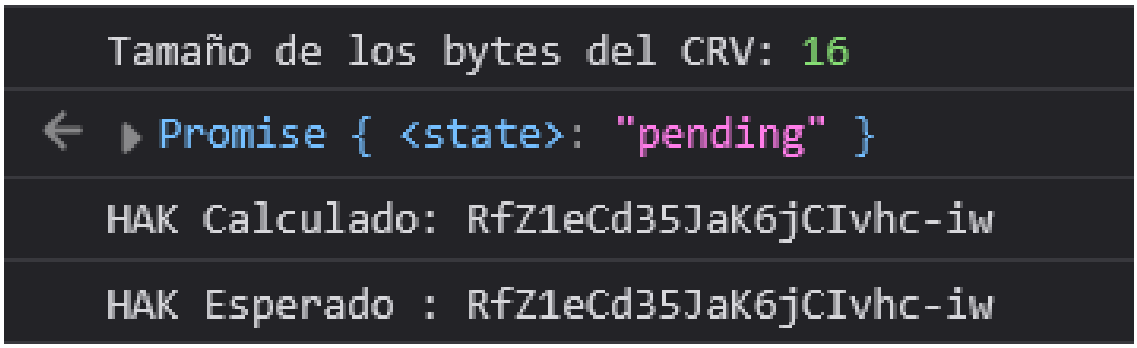


Figura 4: Resultado de la generación del hash de la contraseña en la consola del navegador

El hak esperado es el mismo valor que se observa en la figura 2, y el calculado coincide, así completando el objetivo de este punto.

2.4. Intercepta el tráfico login con BurpSuite

2.5. Realiza el intento de login

2.6. Identifica las políticas de privacidad o seguridad

2.7. Demuestra 4 conclusiones sobre la seguridad